

Polymorphism

The word **Polymorphism** comes from two Greek words:

- **Poly** = Many
- **Morph** = Forms

So, **Polymorphism** means **having many forms**.

In programming, **polymorphism** allows the same function or method to behave differently based on the object or data type that calls it.

Simple Definition

Polymorphism in Python means:

The same function name or operator can perform **different tasks** depending on the type of object it is acting upon.

Real-Life Example

Think of the word “**run**”:

- A human can run (physical action)
- A program can run (execute)
- A machine can run (operate)

The word is the same, but its meaning changes depending on the context. This is exactly what **polymorphism** does in programming.

Types of Polymorphism in Python

1. **Compile-time Polymorphism** (Method Overloading)
2. **Run-time Polymorphism** (Method Overriding)
3. **Operator Overloading**

Python primarily supports **run-time polymorphism** and **operator overloading**, as it doesn't have true compile-time overloading like Java or C++.

1. Method Overriding (Run-time Polymorphism)

In **method overriding**, a child class defines a method that already exists in the parent class, but with a different implementation.

At runtime, Python decides which version of the method to call depending on the object.

Example:

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

class Cat(Animal):
    def speak(self):
        print("Cat meows")
```

Runtime polymorphism

```
animals = [Dog(), Cat(), Animal()]
```

```
for a in animals:
```

```
    a.speak()
```

Output:

Dog barks

Cat meows

Animal makes a sound

Explanation:

- All classes have a `speak()` method.
- The method's behavior depends on which object calls it.
- The same method name works differently for each class.

2. Method Overloading (Compile-time Polymorphism – Simulated in Python)

Python does not support traditional **method overloading** (same method name with different parameters). However, it can be simulated using **default arguments** or ***args**.

Example:

```
class Math:

    def add(self, *args):

        total = 0

        for i in args:

            total += i

        return total
```

```
m = Math()

print(m.add(5))

print(m.add(5, 10))

print(m.add(5, 10, 15))
```

Output:

```
5

15

30
```

Explanation:

Python allows flexible parameters using default values, so one method can handle multiple use cases. This mimics method overloading behavior.

3. Operator Overloading

In Python, operators like +, -, *, etc., can work differently depending on the operands.

Example:

- + adds numbers, but also concatenates strings.

Example 1 (Built-in Overloading):

```
print(10 + 5)    # Adds numbers

print("Aditya " + "Vishwakarma") # Concatenates strings
```

Output:

```
15

Aditya Vishwakarma
```

Example 2 (Custom Operator Overloading):

We can define how operators behave for user-defined classes using **special methods (dunder methods)**.

class Point:

```
def __init__(self, x, y):
```

```
    self.x = x
```

```
    self.y = y
```

```
def __add__(self, other):
```

```
    return Point(self.x + other.x, self.y + other.y)
```

```
def __str__(self):
```

```
    return f"({self.x}, {self.y})"
```

```
p1 = Point(3, 4)
```

```
p2 = Point(1, 2)
```

```
p3 = p1 + p2 # Uses __add__ method
```

```
print(p3)
```

Output:

```
(4, 6)
```

Explanation:

- The + operator works for Point objects because we defined `__add__`.
- You can overload many operators like - (`__sub__`), * (`__mul__`), etc.

4. Polymorphism with Functions and Objects

Python supports polymorphism even with functions — the same function can take objects of different classes and call their methods.

Example:

```
class Bird:
```

```
    def fly(self):
```

```
print("Bird can fly")
```

```
class Airplane:
```

```
    def fly(self):
```

```
        print("Airplane flies in the sky")
```

```
class Fish:
```

```
    def swim(self):
```

```
        print("Fish swims in water")
```

```
def flying_test(entity):
```

```
    entity.fly()
```

```
# Different objects passed to the same function
```

```
bird = Bird()
```

```
plane = Airplane()
```

```
flying_test(bird)
```

```
flying_test(plane)
```

Output:

Bird can fly

Airplane flies in the sky

Explanation:

The same function `flying_test()` works differently depending on the object passed to it. This is a perfect example of polymorphism in action.

Advantages of Polymorphism

Advantage	Description
Code Reusability	The same function can work for different types of objects.

Flexibility	Easier to add new classes or features without modifying existing code.
Readability	Makes the code cleaner and easier to maintain.
Extensibility	New object behaviors can be added without changing existing logic.

Summary Table

Type	Description	Example
Method Overriding	Child class redefines parent class method	Dog.speak()
Method Overloading	Same method with different parameters (simulated)	add(a, b, c=0)
Operator Overloading	Custom meaning for operators	__add__, __mul__
Function Polymorphism	Function acts differently for different objects	flying_test(obj)

Final Summary

- **Polymorphism** means the same name can represent multiple forms.
 - It allows methods, functions, or operators to behave differently depending on the object type.
 - It improves flexibility, readability, and reusability of the code.
-